

Fase 11 — Replicación Lógica Multi-Maestro con MariaDB Galera Cluster

Continuación directa de la Fase 10. `bdd-nodo01` opera como maestro de replicación física y `bdd-nodo02` como esclavo con `read_only = ON`; el snapshot `fase10-completa` fue tomado en ambos nodos. Esta fase introduce la **replicación lógica multi-maestro** mediante **MariaDB Galera Cluster**: los tres nodos se convierten en pares iguales que aceptan lecturas y escrituras simultáneamente con garantías de consistencia síncrona. Se crea `bdd-nodo03`, se instala la librería Galera en los tres nodos, se migra la topología de la Fase 10 y se verifican el comportamiento multi-maestro, la detección automática de conflictos y la recuperación ante la caída de un nodo. Esta fase no introduce fragmentación de datos (eso corresponde a la Fase 13): los tres nodos mantienen una copia completa e idéntica de `lab_bdd`.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Distinguir con precisión la replicación **asíncrona** (Fase 10) de la replicación **síncrona por certificación** de Galera, explicando las implicaciones de cada modelo para consistencia y disponibilidad según el teorema CAP.
2. Explicar qué es un **write-set**, en qué consiste la **certificación** de transacciones en Galera y por qué este mecanismo puede detectar conflictos que la replicación asíncrona no puede.
3. Crear y configurar `bdd-nodo03` a partir de un clon de un nodo existente, ajustando los parámetros que identifican de forma única a cada miembro del clúster.
4. Instalar la librería `galera-4` y definir los parámetros `wsrep` esenciales en los tres nodos del laboratorio.
5. Inicializar un clúster Galera mediante la operación de **bootstrap** y unir los nodos restantes usando SST (*State Snapshot Transfer*).
6. Ejecutar escrituras en **cualquier nodo del clúster** y verificar que se propagan sincrónicamente a los demás nodos, demostrando el comportamiento multi-maestro genuino.
7. Forzar un **escenario de conflicto** (dos transacciones que modifican la misma fila en distintos nodos simultáneamente) y observar cómo Galera resuelve el conflicto con el **rollback automático** de la transacción perdedora.
8. Simular la **caída y recuperación de un nodo**, interpretar los indicadores `wsrep_cluster_size` y `wsrep_cluster_status`, y verificar que el nodo recuperado obtiene los datos que le faltaban sin intervención manual.

9. Cerrar la fase con el snapshot `fase11-completa` en los **tres nodos** en el estado de mayor preparación alcanzado hasta ahora.

B. Conceptos teóricos necesarios

1. Replicación asíncrona vs. síncrona: el compromiso CAP en la práctica.

En la Fase 10 se implementó replicación **asíncrona**: el maestro confirma (`COMMIT`) la transacción al cliente *antes* de que el esclavo la haya aplicado. El esclavo puede quedar rezagado. Si el maestro cae en ese instante, el esclavo puede no tener los últimos cambios — esto es pérdida de datos aceptada a cambio de mayor disponibilidad. Esta es la postura **AP** del teorema CAP: disponibilidad sobre consistencia ante partición.

Galera implementa replicación **síncrona por certificación**: antes de que el `COMMIT` se confirme al cliente, el nodo que originó la transacción envía su *write-set* a todos los demás nodos y espera que todos lo certifiquen. Solo cuando todos confirman que no hay conflicto se confirma la transacción. Esta es la postura **CP**: todos los nodos siempre tienen los mismos datos, pero un nodo que pierde contacto con el clúster deja de aceptar escrituras hasta recuperar la comunicación.

2. El protocolo wsrep y los write-sets.

`wsrep` (*Write-Set REplication*) es una interfaz estándar que MariaDB expone y que la librería Galera implementa. El flujo de una transacción con wsrep es:

Text

```
Cliente → BEGIN en nodo01
Cliente → INSERT / UPDATE / DELETE (se ejecuta localmente en memoria)
Cliente → COMMIT
      ↓
nodo01 serializa los cambios como write-set:
      { primary_keys_afectadas | valores_nuevos | número_de_secuencia_global
      (seqno) }
      ↓
nodo01 difunde el write-set a nodo02 y nodo03 (broadcast)
      ↓
CADA NODO certifica: ¿algún write-set anterior en vuelo toca las
mismas PKs?
      ↓ NO → certificación exitosa → COMMIT aplicado en todos los nodos
      ↓ SÍ → conflicto → el write-set con seqno menor gana;
                el otro recibe ROLLBACK automático
```

La ventaja clave es que la certificación no requiere comunicación durante la ejecución de la transacción — solo en el momento del `COMMIT`. Esto minimiza la penalización de latencia a una sola ronda de red por transacción.

3. SST e IST: cómo un nodo nuevo o recuperado obtiene datos.

Cuando un nodo se une al clúster (o vuelve tras estar offline), necesita sincronizarse:

- **SST (State Snapshot Transfer):** copia completa del estado actual del nodo donante al nodo solicitante (*joiner*). El método `rsync` copia los archivos de datos directamente y es el más simple; `mariabackup` es no bloqueante y se recomienda en producción. Se usa cuando el nodo joiner está completamente desactualizado o es nuevo.
- **IST (Incremental State Transfer):** solo transfiere los write-sets que el joiner no tiene, usando el *gcache* (caché en disco de write-sets recientes). Se usa cuando el nodo estuvo offline poco tiempo y los write-sets faltantes aún están en el gcache. Es mucho más rápido que SST.

El clúster elige automáticamente SST o IST según el estado del nodo que se une.

4. Componente Primario y quórum.

Galera usa el concepto de **Componente Primario (PC)**: la partición del clúster que tiene quórum ($n/2 + 1$ nodos). Solo la PC puede procesar transacciones. Si la red se divide en dos particiones, la que tenga mayoría de nodos continúa como PC; la minoría entra en estado `non-Primary` y rechaza todas las escrituras.

Para un clúster de 3 nodos:

- Quórum = 2 nodos.
- Si cae 1 nodo → los 2 restantes mantienen el quórum y siguen operando.
- Si caen 2 nodos → el nodo restante queda solo, pierde quórum y entra en `non-Primary` (no acepta escrituras). Esto es intencional: se prefiere detener al nodo antes que aceptar datos que podrían no ser consistentes con el resto del clúster cuando este se recupere.

5. El problema del split-brain.

Si una partición de red divide el clúster en dos mitades exactamente iguales, ambas podrían creer que son la PC, aceptar escrituras incompatibles y producir divergencia de datos — el *split-brain*. Un clúster de número **impar** de nodos (3, 5, 7...) nunca produce empate, razón fundamental por la que Galera se opera con 3 nodos mínimo. Un clúster de 2 nodos puede complementarse con un árbitro ligero (`g arb d`) que ocupa el tercer “voto” de quórum sin almacenar datos.

6. `auto_increment` en multi-maestro: prevención de colisiones de clave primaria.

Si dos nodos generan simultáneamente el valor `AUTO_INCREMENT` siguiente, podrían producir el mismo número. Galera resuelve esto con `wsrep_auto_increment_control = ON` (activado por defecto), que ajusta automáticamente dos variables en todos los nodos:

- `auto_increment_increment = N` (donde N = número de nodos del clúster): cada nodo avanza de N en N.
- `auto_increment_offset = [1, 2, 3, ...]` según el número de secuencia del nodo.

Con 3 nodos: nodo01 genera `1, 4, 7, ...`; nodo02 genera `2, 5, 8, ...`; nodo03 genera `3, 6, 9, ...`. Nunca habrá colisiones de PK aunque dos nodos inserten al mismo tiempo.

7. Galera y el binary log: coexistencia con replicación asíncrona downstream.

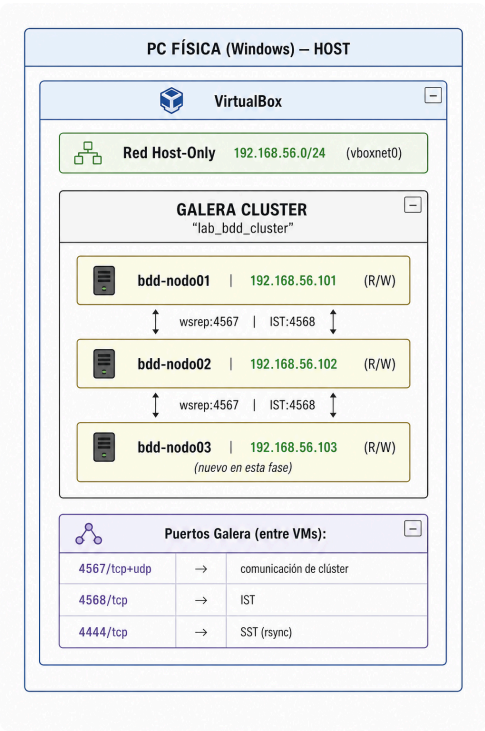
Un clúster Galera puede mantener el binary log activo en uno o todos sus nodos. Esto permite añadir **esclavos asíncronos tradicionales** apuntando a cualquier miembro del clúster — arquitectura conocida como *Galera + async slave* — que se usa para offloading de respaldos o replicación geográfica. En este laboratorio el binary log de nodo01 (activo desde la Fase 10) se conserva con este propósito didáctico.

8. Contraste entre replicación “física” (Fase 10) y “lógica” (Fase 11).

Aspecto	Fase 10 — Física (binlog row-based)	Fase 11 — Lógica (Galera wsrep)
Unidad de replicación	Evento de fila binario (imagen antes/después)	Write-set (PKs afectadas + valores + seqno)
Sincronía	Asíncrona	Síncrona por certificación
Topología	Maestro → esclavo (unidireccional)	Todos los nodos son pares iguales
Escrituras	Solo en el maestro	En cualquier nodo
Detección de conflictos	No (divergencia silenciosa posible)	Automática; rollback determinista
Postura CAP	AP (disponibilidad ante partición)	CP (consistencia ante partición)
Implementación en MariaDB	<code>log_bin</code> , <code>CHANGE MASTER TO</code> , <code>START SLAVE</code>	<code>wsrep_on = ON</code> , <code>galera_new_cluster</code>

C. Arquitectura utilizada

La arquitectura final de esta fase integra tres nodos Galera en la misma red Host-Only:



Nodo	Hostname	IP	Rol en Galera	¿Acepta escrituras?
1	bdd-nodo01	192.168.56.101	Nodo Galera 1 (bootstrapper inicial)	Sí
2	bdd-nodo02	192.168.56.102	Nodo Galera 2 (joiner)	Sí
3	bdd-nodo03	192.168.56.103	Nodo Galera 3 (joiner, nuevo)	Sí

El término “bootstrapper” indica únicamente qué nodo inicializa el cluster en el primer arranque. En operación normal los tres nodos son **pares completamente iguales**: no existe un maestro, no existe un esclavo.

D. Procedimiento paso a paso

Paso 1 — Crear `bdd-nodo03` como clon enlazado de `bdd-nodo01`.

Clonar la VM `bdd-nodo01` desde el snapshot `fase09-completa` (estado previo a la configuración de replicación física), que proporciona una base limpia con Ubuntu Server, MariaDB 10.11 y el esquema `lab_bdd` sin configuración de maestro ni esclavo.

Paso 2 — Personalizar `bdd-nodo03`: `hostname`, `IP` y `server_id`.

Cambiar el `hostname` a `bdd-nodo03`, la IP estática a `192.168.56.103` y el `server_id` de MariaDB a `3`. Actualizar el archivo `/etc/hosts` en los tres nodos para incluir los nombres de los tres miembros.

Paso 3 — Instalar la librería Galera en los tres nodos.

Instalar el paquete `galera-4` que provee la librería `libgalera_smm.so` requerida por MariaDB para el protocolo `wsrep`.

Paso 4 — Limpiar la configuración de esclavo de la Fase 10 en `bdd-nodo02`.

Detener el agente esclavo, ejecutar `RESET SLAVE ALL` y eliminar `read_only = ON` del archivo de configuración. `bdd-nodo02` pasa de esclavo a par Galera.

Paso 5 — Crear el archivo de configuración Galera en cada nodo.

Crear `/etc/mysql/mariadb.conf.d/61-galera.cnf` en los tres nodos con los parámetros `wsrep` necesarios, diferenciando únicamente `wsrep_node_address` y `wsrep_node_name` en cada uno.

Paso 6 — Inicializar el clúster (bootstrap desde `bdd-nodo01`).

Detener MariaDB en `nodo01` y arrancar el clúster con `galera_new_cluster`. En este momento existe un clúster de un solo nodo en estado Primary.

Paso 7 — Unir `bdd-nodo02` al clúster.

Arrancar MariaDB en `nodo02` con `systemctl start mariadb`. Galera detecta el clúster existente, solicita SST y se sincroniza automáticamente.

Paso 8 — Unir `bdd-nodo03` al clúster.

Igual que el paso anterior: arrancar MariaDB en `nodo03` y verificar la sincronización.

Paso 9 — Verificar el estado completo del clúster.

Consultar las variables `wsrep_*` en los tres nodos para confirmar: `wsrep_cluster_size = 3`, `wsrep_cluster_status = Primary`, `wsrep_local_state_comment = Synced` en todos.

Paso 10 — Demostrar escrituras multi-maestro.

Insertar registros en `nodo01` y verificar su presencia inmediata en `nodo02` y `nodo03`. Insertar en `nodo03` y verificar en `nodo01` y `nodo02`.

Paso 11 — Demostrar la detección de conflictos.

Abrir dos sesiones simultáneas, una en nodo01 y otra en nodo03, que modifiquen la misma fila. Observar el rollback automático de la transacción perdedora y verificar que todos los nodos ven el mismo resultado final (determinismo Galera).

Paso 12 — Simular caída y recuperación automática de un nodo.

Detener bruscamente nodo02, insertar datos con el clúster en estado de 2 nodos, reiniciar nodo02 y verificar que recuperó los datos faltantes por IST sin intervención manual.

Paso 13 — Apagar los tres nodos ordenadamente y tomar snapshots.

E. Comandos completos

Los comandos marcados **(host)** se ejecutan en PowerShell en Windows. Los marcados **(VM — nodoXX)** se ejecutan en SSH dentro del nodo indicado. Los bloques SQL dentro de `sudo mariadb` se pueden pegar directamente en el prompt del motor.

E.1 Crear `bdd-nodo03` como clon de `bdd-nodo01` (host)

PowerShell

```
# Verificar que todas las VMs relevantes están apagadas antes de clonar
VBoxManage list runningvms
# Clonar bdd-nodo01 desde el snapshot fase09-completa
# --options link crea un clon enlazado (usa el disco base del snapshot
de origen;
# solo ocupa espacio para los cambios diferenciales). Omitir --options
link para
# un clon completo e independiente (tarda más y ocupa más disco).
VBoxManage clonevm "bdd-nodo01" `
  --snapshot "fase09-completa" `
  --options link `
  --name "bdd-nodo03" `
  --basefolder "C:\LabBDD\VMs" `
  --register
# Confirmar que bdd-nodo03 aparece en la lista de VMs
VBoxManage list vms | Select-String "bdd-nodo03"
# Verificar que el adaptador de red del clon es Host-Only
VBoxManage showvminfo "bdd-nodo03" | Select-String "NIC 1"
# Si el adaptador no está configurado correctamente:
VBoxManage modifyvm "bdd-nodo03" `
  --nic1 hostonly `
  --hostonlyadapter1 "VirtualBox Host-Only Ethernet Adapter"
```

¿Por qué `fase09-completa` y no `fase10-completa` ? El snapshot `fase09-completa` contiene el esquema `lab_bdd` correctamente poblado (Fase 8) sin la configuración de maestro de la Fase 10, lo que da a `nodo03` una base completamente neutra para configurar Galera desde cero.

E.2 Iniciar `bdd-nodo03` y cambiar hostname e IP (host + VM — nodo03)

Arrancar `nodo03` en modo headless:

PowerShell

```
VBoxManage startvm "bdd-nodo03" --type headless
```

El clon hereda la IP `192.168.56.101` de `nodo01`, lo que provoca conflicto de IP. Acceder por la **consola de VirtualBox** (no por SSH todavía): en VirtualBox Manager, seleccionar `bdd-nodo03` → botón **Mostrar** (Show). Iniciar sesión con las credenciales de `bddadmin`.

Dentro de `bdd-nodo03` (consola VirtualBox):

 Bash

```
# ---- Cambiar hostname ----
sudo hostnamectl set-hostname bdd-nodo03
# ---- Identificar nombre de la interfaz de red ----
ip link show
# Normalmente: enp0s3 o enp0s8 (heredado del clon)
# ---- Cambiar la IP estática en Netplan ----
# El nombre del archivo puede variar; listar para encontrarlo:
ls /etc/netplan/
sudo nano /etc/netplan/00-installer-config.yaml
```

Contenido del archivo (ajustar el nombre de interfaz si difiere):

 YAML

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: false
      addresses:
        - 192.168.56.103/24
      routes:
        - to: default
          via: 192.168.56.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Aplicar la configuración de red:

 Bash

```
sudo netplan apply
# Verificar la nueva IP
ip addr show
hostname
```

La interfaz debe mostrar `192.168.56.103` y el hostname debe ser `bdd-nodo03`. Desde este momento SSH funciona desde el host:

 PowerShell

```
# Verificar desde el host
ssh bddadmin@192.168.56.103
```

E.3 Configurar `/etc/hosts` y `server_id` en `bdd-nodo03` (VM — nodo03)

 Bash

```
# Actualizar /etc/hosts para que el nodo conozca a todos los miembros
del clúster
sudo tee /etc/hosts > /dev/null << 'EOF'
127.0.0.1      localhost
127.0.1.1      bdd-nodo03
# Laboratorio BDD — clúster Galera
192.168.56.101 bdd-nodo01
192.168.56.102 bdd-nodo02
192.168.56.103 bdd-nodo03
EOF
# Verificar conectividad de red hacia los otros nodos
ping -c 3 192.168.56.101
ping -c 3 192.168.56.102
```

Cambiar el `server_id` de MariaDB a `3` (debe ser único en el clúster). El archivo de configuración de replicación se creó en la Fase 10; si no existe, crearlo:

 Bash

```
# Verificar si existe el archivo de configuración de la Fase 10
ls /etc/mysql/mariadb.conf.d/60-replication.cnf
# Editar (o crear) el archivo para establecer server_id = 3
sudo nano /etc/mysql/mariadb.conf.d/60-replication.cnf
```

El archivo debe contener como mínimo:

Text

```
[mariadb]
server_id      = 3
log_bin        = /var/log/mysql/mariadb-bin
binlog_format  = ROW
```

Guardar, reiniciar MariaDB y confirmar el `server_id`:

Bash

```
sudo systemctl restart mariadb
sudo mariadb -e "SELECT @@server_id AS server_id;"
```

La salida debe mostrar `3`.

E.4 Actualizar `/etc/hosts` en `bdd-nodo01` y `bdd-nodo02` (VM — nodo01 y nodo02)

Ejecutar en **cada** uno de los dos nodos existentes:

Bash

```
# Ajustar la segunda línea (127.0.1.1) según el hostname del nodo actual
sudo tee /etc/hosts > /dev/null << 'EOF'
127.0.0.1      localhost
127.0.1.1      bdd-nodo01
# Laboratorio BDD — clúster Galera
192.168.56.101 bdd-nodo01
192.168.56.102 bdd-nodo02
192.168.56.103 bdd-nodo03
EOF
```

En `bdd-nodo02`, cambiar la segunda línea a `127.0.1.1 bdd-nodo02`.

Verificar resolución de nombres:

 Bash

```
ping -c 2 bdd-nodo03
```

E.5 Instalar la librería Galera en los tres nodos (VM — nodo01, nodo02, nodo03)

Ejecutar en **cada uno de los tres nodos** (en tres terminales SSH simultáneas):

 Bash

```
# Actualizar lista de paquetes
sudo apt update
# Instalar la librería Galera 4
# En sistemas con MariaDB del repositorio oficial, galera-4 puede
# ya estar instalado como dependencia de mariadb-server.
sudo apt install -y galera-4
# Verificar que la librería existe en la ruta estándar
ls -lh /usr/lib/galera/libgalera_smm.so
```

Si el archivo no está en `/usr/lib/galera/`, ubicarlo con:

 Bash

```
find /usr -name "libgalera_smm.so" 2>/dev/null
```

Registrar la ruta exacta — se usará en el parámetro `wsrep_provider` del siguiente paso.

E.6 Limpiar la configuración de esclavo de la Fase 10 en `bdd-nodo02` (VM — nodo02)

 Bash

```
sudo mariadb << 'EOF'
-- Ver el estado actual del esclavo (documentar antes de eliminar)
SHOW SLAVE STATUS\G
-- Detener el agente de replicación esclavo
STOP SLAVE;
-- Eliminar TODA la configuración de esclavo (master.info, relay-log.info)
RESET SLAVE ALL;
-- Confirmar que ya no hay configuración activa
SHOW SLAVE STATUS\G
EOF
```

La última instrucción `SHOW SLAVE STATUS\G` debe devolver un conjunto vacío.

Eliminar también el parámetro `read_only` del archivo de configuración:

 Bash

```
sudo nano /etc/mysql/mariadb.conf.d/60-replication.cnf
```

Asegurarse de que el archivo contenga únicamente:

 Text

```
[mariadb]
server_id      = 2
log_bin        = /var/log/mysql/mariadb-bin
binlog_format  = ROW
# (sin read_only = ON)
```

| No reiniciar MariaDB todavía; se hará al aplicar la configuración Galera en E.9.

E.7 Crear el archivo de configuración Galera en bdd-nodo01 (VM—nodo01)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster – bdd-nodo01
# Creado en la Fase 11.
# Cargado después de 60-replication.cnf (que ya
# establece binlog_format=ROW).
# =====
[mariadb]
# ---- Activar el protocolo wsrep ----
wsrep_on = ON
wsrep_provider = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options = "pc.recovery=YES; gcache.size=256M"
# ---- Identificación del clúster ----
wsrep_cluster_name = lab_bdd_cluster
wsrep_cluster_address =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Identificación de este nodo ----
wsrep_node_address = 192.168.56.101
wsrep_node_name = bdd-nodo01
# ---- Método de sincronización inicial (SST) ----
# rsync: simple, adecuado para este laboratorio.
# Alternativa de producción: wsrep_sst_method = mariabackup
wsrep_sst_method = rsync
# ---- Hilos para aplicar write-sets recibidos ----
wsrep_slave_threads = 4
# ---- Requerimiento de InnoDB para Galera ----
# innodb_autoinc_lock_mode=2: necesario para auto_increment en
multi-maestro
innodb_autoinc_lock_mode = 2
# ---- Control automático de auto_increment para evitar colisiones de PK -
---
wsrep_auto_increment_control = ON
# Nota: binlog_format = ROW ya está definido en 60-replication.cnf.
# Galera requiere ROW; no duplicar aquí para evitar advertencias
de conflicto.
EOF
# Verificar que no hay errores de sintaxis
cat /etc/mysql/mariadb.conf.d/61-galera.cnf
```

E.8 Crear el archivo de configuración Galera en `bdd-nodo02` (VM — nodo02)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster — bdd-nodo02
# Creado en la Fase 11.
# =====
[mariadb]
wsrep_on                = ON
wsrep_provider           = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options   = "pc.recovery=YES; gcache.size=256M"
wsrep_cluster_name       = lab_bdd_cluster
wsrep_cluster_address    =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Solo estas dos líneas difieren de nodo01 ----
wsrep_node_address       = 192.168.56.102
wsrep_node_name          = bdd-nodo02
wsrep_sst_method         = rsync
wsrep_slave_threads      = 4
innodb_autoinc_lock_mode = 2
wsrep_auto_increment_control = ON
EOF
```

E.9 Crear el archivo de configuración Galera en `bdd-nodo03` (VM — nodo03)

 Bash

```
sudo tee /etc/mysql/mariadb.conf.d/61-galera.cnf > /dev/null << 'EOF'
# =====
# Configuración Galera Cluster — bdd-nodo03
# Creado en la Fase 11.
# =====
[mariadb]
wsrep_on                = ON
wsrep_provider           = /usr/lib/galera/libgalera_smm.so
wsrep_provider_options   = "pc.recovery=YES; gcache.size=256M"
wsrep_cluster_name       = lab_bdd_cluster
wsrep_cluster_address    =
gcomm://192.168.56.101,192.168.56.102,192.168.56.103
# ---- Solo estas dos líneas difieren de nodo01 ----
wsrep_node_address       = 192.168.56.103
wsrep_node_name          = bdd-nodo03
wsrep_sst_method         = rsync
wsrep_slave_threads      = 4
innodb_autoinc_lock_mode = 2
wsrep_auto_increment_control = ON
EOF
```

E.10 Verificar y abrir puertos Galera si el firewall está activo (VM — los tres nodos)

 Bash

```
# Comprobar si ufw está activo (Ubuntu 24.04 lo instala pero inactivo
por defecto)
sudo ufw status
# Si el resultado es "Status: inactive" no se necesita hacer nada.
# Si está activo, abrir los puertos Galera en CADA nodo:
sudo ufw allow 3306/tcp    comment 'MariaDB cliente'
sudo ufw allow 4567/tcp    comment 'Galera cluster wsrep'
sudo ufw allow 4567/udp    comment 'Galera cluster wsrep UDP'
sudo ufw allow 4568/tcp    comment 'Galera IST'
sudo ufw allow 4444/tcp    comment 'Galera SST rsync'
sudo ufw reload
```

E.11 Inicializar el clúster — Bootstrap desde `bdd-nodo01` (VM — nodo01)

 Bash

```
# Detener el servicio MariaDB estándar
sudo systemctl stop mariadb
# Inicializar el clúster Galera.
# galera_new_cluster arranca MariaDB con wsrep_cluster_address='gcomm://'
# (cadena vacía), señal para Galera de que este nodo es el primero
del clúster
# y debe inicializarlo con un nuevo UUID de clúster.
sudo galera_new_cluster
# Verificar que el servicio arrancó correctamente
sudo systemctl status mariadb --no-pager
```

Verificar el estado del clúster en nodo01 (en este punto solo hay 1 nodo):

 Bash

```
sudo mariadb -e "
SHOW STATUS WHERE Variable_name IN (
    'wsrep_cluster_size',
    'wsrep_cluster_status',
    'wsrep_connected',
    'wsrep_ready',
    'wsrep_local_state_comment',
    'wsrep_node_name'
);
"
```

Salida esperada:

Text

Variable_name	Value
wsrep_cluster_size	1
wsrep_cluster_status	Primary
wsrep_connected	ON
wsrep_ready	ON
wsrep_local_state_comment	Synced
wsrep_node_name	bdd-nodo01

Alternativa:

SQL

```
SELECT 'wsrep_node_name' AS Variable_name,
       @@wsrep_node_name AS Value
UNION ALL
SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM information_schema.GLOBAL_STATUS
WHERE VARIABLE_NAME IN (
    'WSREP_CLUSTER_SIZE',
    'WSREP_CLUSTER_STATUS',
    'WSREP_CONNECTED',
    'WSREP_READY',
    'WSREP_LOCAL_STATE_COMMENT'
);
```

Salida esperada:

Text

Variable_name	Value
wsrep_node_name	bdd-nodo01
WSREP_LOCAL_STATE_COMMENT	Synced
WSREP_CLUSTER_SIZE	1
WSREP_CLUSTER_STATUS	Primary
WSREP_CONNECTED	ON
WSREP_READY	ON

`wsrep_cluster_size = 1` es correcto en este punto: solo nodo01 está en el clúster. Un valor de `wsrep_cluster_status = Primary` con un solo nodo es normal y esperado durante el bootstrap.

E.12 Unir `bdd-nodo02` al clúster (VM — nodo02)

Bash

```
# Arrancar MariaDB normalmente (NO usar galera_new_cluster -
# ese comando crea un clúster nuevo; aquí queremos UNIRSE al existente).
sudo systemctl start mariadb
# Observar el log en tiempo real durante el SST (puede tardar 30-120 s)
sudo journalctl -u mariadb -f --no-pager | grep -
E "WSREP|SST|Synced|state"
# Ctrl+C para salir cuando aparezca "WSREP: Synchronized with group"
```

Verificar que nodo02 se unió correctamente:

 Bash

```
sudo mariadb -e "  
SHOW STATUS WHERE Variable_name IN (  
    'wsrep_cluster_size',  
    'wsrep_cluster_status',  
    'wsrep_local_state_comment',  
    'wsrep_node_name'  
);  
"
```

Salida esperada en nodo02:

 Text

```
+-----+-----+  
| Variable_name          | Value          |  
+-----+-----+  
| wsrep_cluster_size     | 2              |  
| wsrep_cluster_status   | Primary        |  
| wsrep_local_state_comment | Synced         |  
| wsrep_node_name        | bdd-nodo02     |  
+-----+-----+
```

Confirmar desde nodo01 que el clúster creció:

 Bash

```
# En bdd-nodo01  
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
```

E.13 Unir `bdd-nodo03` al clúster (VM — nodo03)

 Bash

```
sudo systemctl start mariadb  
# Monitorear el progreso del SST  
sudo journalctl -u mariadb -f --no-pager | grep -E "WSREP|SST|IST|Synced"  
# Ctrl+C al aparecer "Synchronized with group"
```

Verificar el estado final:

 Bash

```
sudo mariadb -e "  
SHOW STATUS WHERE Variable_name IN (  
    'wsrep_cluster_size',  
    'wsrep_cluster_status',  
    'wsrep_local_state_comment',  
    'wsrep_node_name'  
);  
"
```

Salida esperada en nodo03:

 Text

```
+-----+-----+  
| Variable_name          | Value          |  
+-----+-----+  
| wsrep_cluster_size     | 3              |  
| wsrep_cluster_status   | Primary        |  
| wsrep_local_state_comment | Synced         |  
| wsrep_node_name        | bdd-nodo03     |  
+-----+-----+
```

E.14 Verificación completa del estado del clúster en los tres nodos (VM)

Ejecutar en **cada uno de los tres nodos**:

 Bash

```
sudo mariadb << 'EOF'
-- Estado global del clúster
SELECT VARIABLE_NAME, VARIABLE_VALUE AS valor
FROM information_schema.GLOBAL_STATUS
WHERE VARIABLE_NAME IN (
    'WSREP_CLUSTER_SIZE',
    'WSREP_CLUSTER_STATUS',
    'WSREP_CONNECTED',
    'WSREP_READY',
    'WSREP_LOCAL_STATE_COMMENT',
    'WSREP_NODE_NAME',
    'WSREP_FLOW_CONTROL_PAUSED',
    'WSREP_CERT_DEPS_DISTANCE'
)
ORDER BY VARIABLE_NAME;
-- Miembros actuales del clúster (tabla disponible solo con Galera activo)
SELECT * FROM information_schema.WSREP_MEMBERSHIP
ORDER BY INDEX;
-- Verificar que auto_increment fue ajustado automáticamente por Galera
SHOW VARIABLES LIKE 'auto_increment_increment';
SHOW VARIABLES LIKE 'auto_increment_offset';
-- Confirmar que lab_bdd tiene los datos correctos
SELECT 'clientes' AS tabla, COUNT(*) AS registros FROM
lab_bdd.clientes UNION ALL
SELECT 'productos', COUNT(*) FROM
lab_bdd.productos UNION ALL
SELECT 'pedidos', COUNT(*) FROM
lab_bdd.pedidos UNION ALL
SELECT 'detalle_pedidos', COUNT(*) FROM
lab_bdd.detalle_pedidos;
EOF
```

`information_schema.WSREP_MEMBERSHIP` debe mostrar exactamente 3 filas con los nombres `bdd-nodo01`, `bdd-nodo02`, `bdd-nodo03`, sus IPs y estado `Synced`. Esta tabla solo existe cuando Galera está activo.

E.15 Demostrar escrituras multi-maestro (VM — nodo01 y nodo03)

Prueba A: escritura en nodo01, lectura inmediata en nodo02 y nodo03.

En `bdd-nodo01`:

 Bash

```
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'GaleraA', 'prueba.galera.a@lab.test',
        '5500000021', 'norte', 'Ciudad de México');
SELECT id, nombre, apellido, email FROM clientes
WHERE email = 'prueba.galera.a@lab.test';
EOF
```

Verificar inmediatamente en nodo02 y nodo03 (la replicación es síncrona — no hay lag):

 Bash

```
# Ejecutar en bdd-nodo02 Y en bdd-nodo03
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email, ciudad
FROM clientes WHERE email = 'prueba.galera.a@lab.test';
"
```

El registro debe aparecer en los tres nodos con el **mismo** `id`.

Prueba B: escritura directamente en nodo03.

 Bash

```
# En bdd-nodo03
sudo mariadb lab_bdd << 'EOF'
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'GaleraC', 'prueba.galera.c@lab.test',
        '5500000022', 'sur', 'Guadalajara');
SELECT id, nombre, apellido, email FROM clientes
WHERE email = 'prueba.galera.c@lab.test';
EOF
```

Verificar en nodo01 y nodo02:

 Bash

```
# En bdd-nodo01 Y en bdd-nodo02
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email
FROM clientes WHERE email LIKE 'prueba.galera.%'
ORDER BY id;
"
```

Ambos registros (`.a` y `.c`) deben aparecer en los tres nodos. Verificar también el incremento automático de PK por nodo:

 Bash

```
# En cualquier nodo
sudo mariadb -e "
SHOW VARIABLES LIKE 'auto_increment_increment';
SHOW VARIABLES LIKE 'auto_increment_offset';
"
```

Con un clúster de 3 nodos y `wsrep_auto_increment_control = ON` , la salida típica muestra `auto_increment_increment = 3` . El offset varía: `1` en nodo01, `2` en nodo02, `3` en nodo03.

E.16 Demostrar detección de conflictos (VM — nodo01 y nodo03 simultáneamente)

Este experimento requiere **dos terminales SSH abiertas al mismo tiempo**: una a nodo01 y otra a nodo03.

Preparar la fila de prueba:

 Bash

```
# En cualquier nodo — identificar la fila que se usará
sudo mariadb lab_bdd -e "SELECT id, nombre, ciudad FROM clientes WHERE id
= 1;"
```

Terminal A — sesión en nodo01 (iniciar primero):

 Bash

```
sudo mariadb lab_bdd
```

Dentro del prompt de MariaDB en nodo01:

 SQL

```
-- La transacción modifica la fila id=1 localmente y luego espera
10 segundos
-- antes de intentar COMMIT; ese tiempo es la ventana para que
nodo03 cometa.
START TRANSACTION;
UPDATE clientes SET ciudad = 'Ciudad_desde_nodo01' WHERE id = 1;
SELECT SLEEP(10);
COMMIT;
```

Terminal B — sesión en nodo03 (ejecutar MIENTRAS el SLEEP corre en Terminal A):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- Nodo03 modifica la misma fila y confirma ANTES de que nodo01 salga
del SLEEP
START TRANSACTION;
UPDATE clientes SET ciudad = 'Ciudad_desde_nodo03' WHERE id = 1;
COMMIT;
-- Esta transacción debe TENER ÉXITO (se certifica primero)
SELECT 'Transaccion nodo03: EXITOSA' AS resultado;
EOF
```

Resultado esperado: la sesión en nodo01 recibirá un error similar a:

 Text

```
ERROR 1213 (40001): Deadlock found when trying to get lock;
try restarting transaction
```

Verificar que todos los nodos tienen el **mismo valor final** (determinismo de Galera):

 Bash

```
# En los TRES nodos - deben devolver EXACTAMENTE el mismo resultado
sudo mariadb lab_bdd -e "SELECT id, nombre, ciudad FROM clientes WHERE id
= 1;"
```

Solo uno de los dos valores (`Ciudad_desde_nodo01` o `Ciudad_desde_nodo03`) estará presente, y será el **mismo en los tres nodos**. Este es el determinismo de Galera: el conflicto se resuelve de forma consistente globalmente.

Restaurar el valor original:

 Bash

```
sudo mariadb lab_bdd -e "UPDATE clientes SET ciudad = 'Monterrey' WHERE id
= 1;"
```

E.17 Simular caída y recuperación automática de un nodo (host + VM)

Paso 1: Verificar estado inicial (3 nodos).

 Bash

```
# En cualquier nodo
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
# → debe devolver 3
```

Paso 2: Simular caída abrupta de `bdd-nodo02` .

 PowerShell

```
# Desde el host Windows (simula fallo de hardware, sin apagado limpio)
VBoxManage controlvm "bdd-nodo02" poweroff
```

Paso 3: Verificar que el clúster sigue operando con 2 nodos.

 Bash

```
# En bdd-nodo01 o bdd-nodo03
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"      # → 2
sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_status';"    # → Primary
```

Paso 4: Insertar datos mientras nodo02 está caído.

 Bash

```
# En bdd-nodo01
sudo mariadb lab_bdd -e "
INSERT INTO clientes (nombre, apellido, email, telefono, region, ciudad)
VALUES ('Prueba', 'Recuperacion', 'prueba.recovery@lab.test',
        '5500000023', 'este', 'Veracruz');
SELECT 'Registro insertado con nodo02 caido' AS mensaje,
       LAST_INSERT_ID() AS id_asignado;
"
```

Paso 5: Reiniciar `bdd-nodo02` y observar la rejoinura automática.

 PowerShell

```
# Desde el host
VBoxManage startvm "bdd-nodo02" --type headless
```

Esperar 30–60 segundos. Monitorear los logs de nodo02:

 Bash

```
# En bdd-nodo02 (cuando SSH esté disponible)
sudo journalctl -u mariadb -f --no-pager | grep -E
"WSREP|IST|SST|Synced|joining"
# Buscar: "WSREP: Synchronized with group, ready for connections"
```

Paso 6: Verificar que nodo02 recuperó el registro insertado durante la caída.

 Bash

```
# En bdd-nodo02
sudo mariadb lab_bdd -e "
SELECT id, nombre, apellido, email, ciudad
FROM clientes WHERE email = 'prueba.recovery@lab.test';
"

sudo mariadb -e "SHOW STATUS LIKE 'wsrep_cluster_size';"
# → debe volver a 3
```

E.18 Apagar los tres nodos ordenadamente y tomar snapshots (host + VM)

El orden de apagado de un clúster Galera importa: apagar los joiners primero y el bootstrapper (o el nodo con mayor `seqno`) al final. El último en apagarse limpiamente tendrá

`safe_to_bootstrap: 1` en su archivo `grastate.dat`.

 Bash

```
# En bdd-nodo03 (apagar primero)
sudo systemctl stop mariadb && sudo poweroff
# En bdd-nodo02 (apagar segundo)
sudo systemctl stop mariadb && sudo poweroff
# En bdd-nodo01 (apagar ÚLTIMO)
sudo systemctl stop mariadb && sudo poweroff
```

Confirmar desde el host:

 PowerShell

```
VBoxManage list runningvms
# La salida debe estar vacía
```

Tomar los snapshots de cierre de fase:

PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase11-completa" `
--description "Galera Cluster activo (nodo01 bootstrapper). lab_bdd con 22
clientes. wsrep_cluster_size=3, status=Primary. Binary log conservado para
async downstream."
VBoxManage snapshot "bdd-nodo02" take "fase11-completa" `
--description "Galera Cluster miembro. Configuración esclavo Fase10
eliminada. Sincronizado via SST. lab_bdd verificada."
VBoxManage snapshot "bdd-nodo03" take "fase11-completa" `
--description "Tercer miembro Galera. Creado como clon de nodo01@fase09-
completa. IP .103, server_id=3. lab_bdd sincronizada por Galera."
```

Confirmar los snapshots:

PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
VBoxManage snapshot "bdd-nodo03" list
```

`bdd-nodo01` y `bdd-nodo02` deben mostrar los snapshots `fase05-completa` a `fase11-completa`. `bdd-nodo03` debe mostrar únicamente `fase11-completa` (su primer snapshot).

F. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes con el clúster activo:

1. Los tres nodos responden a `ssh bddadmin@192.168.56.10X` desde el host Windows sin errores.
2. `SHOW STATUS LIKE 'wsrep_cluster_size';` devuelve **3** en los **tres nodos**.
3. `SHOW STATUS LIKE 'wsrep_cluster_status';` devuelve **Primary** en los **tres nodos**.
4. `SHOW STATUS WHERE Variable_name = 'wsrep_local_state_comment';` devuelve **Synced** en los **tres nodos**.
5. `SELECT * FROM information_schema.WSREP_MEMBERSHIP;` muestra exactamente 3 filas con los nombres `bdd-nodo01`, `bdd-nodo02`, `bdd-nodo03` y sus IPs correctas.
6. El conteo de registros en `lab_bdd` es **idéntico** en los tres nodos: ≥22 clientes (20 originales + los insertados en las pruebas), 10 productos, 20 pedidos, 36 detalles.
7. Un `INSERT` ejecutado en nodo01 es **inmediatamente visible** en nodo02 y nodo03 sin ningún comando adicional.

8. Un `INSERT` ejecutado en nodo03 es **inmediatamente visible** en nodo01 y nodo02.
 9. El experimento de conflicto (E.16) produjo exactamente **un error** en una de las dos sesiones y el valor de la columna `ciudad` del cliente `id = 1` es **el mismo** en los tres nodos (resultado determinista).
 10. `SHOW VARIABLES LIKE 'auto_increment_increment';` devuelve **3** en los tres nodos.
 11. El experimento de caída de nodo02 (E.17) mostró: `wsrep_cluster_size = 2` durante la caída; `wsrep_cluster_status = Primary` con 2 nodos; el registro `prueba.recovery@lab.test` aparece en nodo02 tras su reinicio; `wsrep_cluster_size = 3` tras la recuperación.
 12. `SHOW VARIABLES LIKE 'wsrep_on';` devuelve **ON** en los tres nodos.
 13. Los snapshots `fase11-completa` existen en los **tres nodos**.
 14. El estudiante puede explicar de memoria la diferencia entre SST e IST y cuándo Galera elige cada mecanismo.
 15. El estudiante puede explicar qué es el Componente Primario y qué ocurre con un nodo que pierde quórum.
-

G. Problemas comunes y soluciones

Problema	Causa probable	Solución
<pre>galera_new_cluster</pre> falla con <code>ERROR: MySQL is running, please stop it first</code>	MariaDB sigue activo	Ejecutar <code>sudo systemctl stop mariadb</code> y luego <code>sudo galera_new_cluster</code>
El nodo joiner permanece en <code>wsrep_local_state_comment = Joining</code> sin avanzar	Falla de conectividad en el puerto 4567 entre los nodos	Verificar <code>ping</code> entre VMs; si ufw está activo, ejecutar E.10; probar con <code>nc -zv 192.168.56.101 4567</code> desde el joiner
El SST tarda más de 5 minutos	Con <code>rsync</code> , el SST es una copia completa; con mucha actividad en el donador puede ser lento	Es normal; esperar. Para datos crecientes, cambiar a <code>wsrep_sst_method = mariabackup</code> (requiere instalar <code>mariadb-backup</code>)
<pre>galera_new_cluster</pre> muestra <code>WSREP: It may not be safe to bootstrap...</code>	Galera detecta datos de un clúster anterior y teme un split-brain	Verificar que los otros nodos están completamente detenidos; editar <code>/var/lib/mysql/grastate.dat</code> en nodo01 y cambiar <code>safe_to_bootstrap: 0</code> a <code>safe_to_bootstrap: 1</code> , luego reintentar

MariaDB no arranca después de agregar <code>61-galera.cnf</code>	Error de sintaxis en el archivo o conflicto de parámetros entre archivos	Revisar el error con <code>sudo journalctl -u mariadb -n 30 --no-pager</code> ; buscar <code>unknown variable</code> o <code>conflicts</code> with para identificar el parámetro problemático
<code>wsrep_cluster_size</code> sigue en 1 después de unir el segundo nodo	Los nodos no se alcanzan en el puerto wsrep 4567 aunque SSH (22) sí funcione	Verificar que <code>wsrep_node_address</code> en <code>61-galera.cnf</code> coincide exactamente con la IP de la interfaz Host-Only; revisar ufw
El experimento de conflicto no genera error: ambas transacciones confirman	El SLEEP fue insuficiente; nodo01 confirmó antes de que nodo03 empezara	Aumentar el SLEEP a 15 segundos; asegurarse de que la sesión en nodo03 inició el UPDATE ANTES de que nodo01 llamara al COMMIT
<code>wsrep_cluster_status = non-Primary</code> en un nodo tras reinicio	El nodo perdió contacto con la mayoría del clúster	Si los otros dos nodos están <code>Primary</code>, el nodo aislado se reconectará automáticamente e al recuperar la red
Tras una caída total (todos los nodos apagados sin orden), ningún nodo arranca el clúster	Galera no sabe cuál nodo tiene el estado más reciente	Revisar <code>/var/lib/mysql/grastate.dat</code> en cada nodo; el que tenga <code>seqno</code> más alto Y

		<pre>safe_to_bootstrap: 1 debe ejecutar galera_new_cluster ; los demás luego con systemctl start mariadb</pre>
El clon de nodo03 tiene el mismo <code>/etc/machine-id</code> que nodo01	VirtualBox regenera el UUID de VM pero no el ID del sistema operativo	En nodo03: <pre>sudo rm /etc/machine-id && sudo systemd-machine-id-setup y reiniciar</pre>
<code>auto_increment_increment</code> sigue en 1 después de activar Galera	<code>wsrep_auto_increment_control = ON</code> no está en <code>61-galera.cnf</code> o el archivo no se cargó	Verificar con <pre>SHOW VARIABLES LIKE 'wsrep_auto_increment_control';</pre> si devuelve <code>OFF</code> , agregar el parámetro al archivo y reiniciar MariaDB

H. Checklist de validación

- ☐ `bdd-nodo03` existe en VirtualBox con IP `192.168.56.103`, hostname `bdd-nodo03` y acceso SSH desde el host.
- ☐ `bdd-nodo03` tiene `server_id = 3` confirmado con `SELECT @@server_id;`.
- ☐ El paquete `galera-4` está instalado y `/usr/lib/galera/libgalera_smm.so` existe en los **tres nodos**.
- ☐ `/etc/mysql/mariadb.conf.d/61-galera.cnf` existe en los **tres nodos** con `wsrep_node_address` y `wsrep_node_name` únicos para cada nodo.
- ☐ La configuración de esclavo de la Fase 10 fue eliminada de `bdd-nodo02`: `RESET SLAVE ALL` ejecutado y `read_only` eliminado del archivo de configuración.
- ☐ El bootstrap se completó en `bdd-nodo01` sin errores y el servicio `mariadb` está activo.

- ☐ `bdd-nodo02` y `bdd-nodo03` se unieron al clúster; sus logs muestran `Synchronized with group`.
- ☐ `SHOW STATUS LIKE 'wsrep_cluster_size';` devuelve **3** en los **tres nodos**.
- ☐ `SHOW STATUS LIKE 'wsrep_cluster_status';` devuelve **Primary** en los **tres nodos**.
- ☐ `wsrep_local_state_comment = Synced` en los **tres nodos**.
- ☐ `information_schema.WSREP_MEMBERSHIP` muestra 3 filas con nodos, IPs y nombres correctos.
- ☐ El conteo de registros en `lab_bdd` es idéntico en los **tres nodos**.
- ☐ Se demostró inserción en nodo01 visible inmediatamente en nodo02 y nodo03.
- ☐ Se demostró inserción en nodo03 visible inmediatamente en nodo01 y nodo02.
- ☐ El experimento de conflicto generó exactamente un error en una de las dos sesiones.
- ☐ El valor del cliente `id = 1` es idéntico en los tres nodos tras el experimento de conflicto.
- ☐ El experimento de caída de nodo02 mostró `wsrep_cluster_size = 2` durante la caída y **3** tras la recuperación.
- ☐ El registro `prueba.recovery@lab.test` aparece en nodo02 tras su reinicio, sin intervención manual.
- ☐ `SHOW VARIABLES LIKE 'auto_increment_increment';` devuelve **3** en los tres nodos.
- ☐ Los snapshots `fase11-completa` existen en los **tres nodos**.
- ☐ Puedo explicar la diferencia entre SST e IST y cuándo Galera elige cada uno.
- ☐ Puedo explicar qué es el Componente Primario y qué ocurre cuando un nodo pierde quórum.

Preguntas teóricas para estudiantes

1. La replicación de Galera se denomina “síncrona por certificación” pero **no** usa commit de dos fases clásico (2PC) ni locks distribuidos durante la ejecución de la transacción. Explica en detalle: ¿qué información contiene exactamente un *write-set*?, ¿qué condición verifica cada nodo durante la certificación?, y ¿por qué este mecanismo puede garantizar consistencia global sin necesitar que los nodos se comuniquen durante la ejecución de la transacción (solo al momento del `COMMIT`)?
2. Un clúster de 3 nodos tolera la caída de 1 nodo simultáneo. Si el sistema de producción necesitara tolerar 2 nodos caídos simultáneamente, ¿cuántos nodos totales se requerirían y por qué? Generaliza la fórmula para una tolerancia a `f` fallos y explica por qué Galera (y cualquier sistema basado en quórum) opera correctamente con número impar de nodos pero tiene problemas estructurales con número par.

3. En el experimento de conflicto (sección E.16), la transacción perdedora recibió un error de tipo *deadlock*. Desde el punto de vista del código de una aplicación que usa este clúster, ¿qué estrategia de manejo de errores es necesaria para que las escrituras sean robustas ante conflictos de certificación? ¿Es suficiente capturar el error y reintentar sin más? ¿Qué cuidados especiales hay que tener en el reintento para no violar la lógica de negocio?
 4. `wsrep_auto_increment_control = ON` hace que Galera ajuste `auto_increment_increment` y `auto_increment_offset` según el número de nodos. ¿Qué ocurre internamente cuando un cuarto nodo se une al clúster en caliente (sin reiniciar los otros nodos)? ¿cómo cambian estos valores en todos los nodos?, ¿qué sucede con las filas ya insertadas que tienen IDs generados con el patrón de 3 nodos?, ¿produce esto algún hueco en la secuencia y es eso un problema?
 5. Dado el diseño completo del laboratorio (Galera en nodo01–03, fragmentación en nodo04–05, coordinador Spider en nodo06), identifica **dos escenarios de partición de red distintos** y explica para cada uno qué dilema CAP presenta: ¿qué parte del sistema elegirá Consistencia (dejando de aceptar escrituras) y qué parte elegirá Disponibilidad (aceptando escrituras con riesgo de inconsistencia)?
-

Ejercicios prácticos

1. Comparación de latencia: asíncrona (Fase 10) vs. síncrona (Galera).

Usando los snapshots `fase10-completa` de nodo01 y nodo02 (desde una sesión VirtualBox paralela, sin afectar el clúster activo), diseñar un experimento que inserte 100 filas en el maestro de la Fase 10 y mida el tiempo promedio entre el `COMMIT` en el maestro y la disponibilidad del dato en el esclavo (via `SELECT COUNT(*)` iterativo). Realizar la misma medición en el clúster Galera activo. Comparar los resultados y explicar qué significa la diferencia para una aplicación que necesita leer inmediatamente después de escribir.

2. Script de monitoreo de salud del clúster Galera.

Escribir un script Bash (`/opt/lab_bdd_backups/galera_health.sh`) que se conecte a MariaDB y genere un reporte con: tamaño del clúster, estado de cada nodo (`wsrep_local_state_comment`), bytes replicados acumulados, transacciones en cola (`wsrep_local_send_queue`) y un semáforo basado en las condiciones: **verde** = 3 nodos Primary Synced; **amarillo** = 2 nodos Primary; **rojo** = menos de 2 nodos o estado non-Primary. El script debe poder ejecutarse desde cualquier nodo e indicar desde cuál nodo reporta.

3. Análisis del archivo `grastate.dat` antes y después de un apagado ordenado.

Antes de apagar el clúster (con `sudo systemctl stop mariadb`), registrar el contenido de `/var/lib/mysql/grastate.dat` en los tres nodos. Luego apagar en el orden correcto (nodo03, nodo02, nodo01) y volver a leer el archivo en cada nodo. Comparar los valores

de `uuid`, `seqno` y `safe_to_bootstrap` antes y después del apagado. Documentar qué nodo tiene `safe_to_bootstrap: 1` y explicar el procedimiento correcto para recuperar el clúster tras una caída total inesperada (todos los nodos apagados de golpe).

Reto adicional para alumnos avanzados

Configurar un **esclavo asíncrono tradicional** apuntando a `bdd-nodo01` (miembro del clúster Galera) desde un cuarto nodo hipotético creado como clon limpio. Las escrituras al clúster se harían con las garantías síncronas de Galera, y la réplica asíncrona serviría para offloading de respaldos. El reto requiere: (a) investigar y documentar el problema de los **GTIDs de Galera** (`wsrep_gtid_mode`) vs los GTIDs de MariaDB y cómo coexisten; (b) configurar nodo01 para emitir binary log con una configuración compatible; © verificar que una escritura en cualquier nodo del clúster llega eventualmente al esclavo async; y (d) elaborar un diagrama de flujo completo de una transacción desde el cliente hasta el esclavo async, indicando en qué puntos la propagación es síncrona y en cuáles es asíncrona, y qué datos podría perder el esclavo async en caso de fallo de nodo01 que no perdería ningún miembro Galera.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación y configuración de <code>bdd-nodo03</code>	10%	El nodo existe con IP <code>.103</code> , hostname correcto, <code>server_id = 3</code> y <code>61-galera.cnf</code> con parámetros únicos; clon tomado de <code>fase09-completa</code>
Configuración Galera en los tres nodos	20%	<code>61-galera.cnf</code> existe en los tres nodos sin errores de sintaxis; MariaDB arranca con Galera habilitado; limpieza de esclavo de Fase 10 realizada en nodo02
Estado del clúster verificado	25%	<code>wsrep_cluster_size = 3</code> , <code>wsrep_cluster_status = Primary</code> y <code>wsrep_local_state_comment = Synced</code> en los tres nodos simultáneamente; <code>WSREP_MEMBERSHIP</code> <code>IP</code> muestra los tres nodos
Demostración multi-maestro y conflictos	20%	Se demuestra escritura exitosa desde nodo01 visible en

		nodo03, y desde nodo03 visible en nodo01; el experimento de conflicto produce exactamente un rollback con resultado idéntico en los tres nodos
Caída y recuperación de nodo	15%	<code>wsrep_cluster_size = 2</code> durante la caída, <code>= 3</code> tras la recuperación; el registro insertado durante la caída aparece en nodo02 sin intervención manual
Comprensión conceptual (preguntas teóricas)	10%	Las respuestas distinguen correctamente la certificación Galera del 2PC clásico, calculan nodos necesarios para tolerancia a fallos y discuten la estrategia de reintento con vocabulario técnico apropiado

Preparación para la siguiente fase

La **Fase 12: Particionamiento** (técnica de organización interna de datos dentro de un nodo, puente conceptual hacia la fragmentación horizontal de la Fase 13) requerirá:

- El clúster Galera de tres nodos en estado `fase11-completa` (esta fase).

- El esquema `lab_bdd` con datos de prueba intactos y consistentes en los tres nodos.
- Acceso SSH y DBeaver funcionando hacia los tres nodos (nodo03 se agrega ahora como cuarta conexión con túnel SSH, siguiendo el mismo procedimiento de la Fase 8).
- Comprensión clara de la diferencia entre **particionamiento** (dividir filas dentro de un mismo nodo usando la directiva `PARTITION BY` de MariaDB, sin mover datos entre servidores) y **fragmentación** (distribuir datos entre nodos distintos, que se implementa en la Fase 13). Esa distinción es el hilo conductor de la Fase 12.

Cualquier operación DDL (`CREATE TABLE` , `ALTER TABLE ... PARTITION BY`) que se ejecute en un nodo del clúster Galera se propagará automáticamente a los otros dos, lo que permite verificar que el DDL de particionamiento se replica correctamente — una verificación adicional de que el clúster sigue funcionando bien después de esta fase.
